МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 8

Тема: Поведенческие паттерны: Цепочка обязанностей

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы поведенческого паттерна проектирования «Цепочка обязанностей» (Chain of Responsibility) и реализовать прикладную задачу «Графический пароль» с использованием данного паттерна.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать поведенческий паттерн «Цепочка обязанностей».

В соответствии с заданием был выбран паттерн «Цепочка обязанностей» (Chain of Responsibility).

Вариант 8 - Графический пароль

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, а также демонстрирует работу паттерна «Цепочка обязанностей»: валидация пароля разбита на независимые обработчики, каждый из которых проверяет одно правило.

1. Описание паттерна «Цепочка обязанностей»

Паттерн «Цепочка обязанностей» (Chain of Responsibility) - это поведенческий паттерн проектирования, который позволяет передавать запрос последовательно по цепочке обработчиков. Каждый обработчик решает, обработать запрос самому или передать его следующему обработчику в цепочке.

Данный паттерн применяется в ситуациях, когда нужно ослабить связность между отправителем запроса и получателями, когда запрос должен быть обработан одним из нескольких объектов, или когда набор обработчиков может изменяться динамически.

В разработанном приложении паттерн «Цепочка обязанностей» используется для валидации последовательности графического пароля. В программе определены следующие роли:

- SequenceValidator - абстрактный обработчик с методами set_next(handler) и handle(sequence);
- LengthValidator - проверяет длину последовательности (4–9 ячеек);

- DuplicateValidator - проверяет отсутствие повторяющихся ячеек;

- RangeValidator - проверяет, что все ячейки находятся в диапазоне 1–9;

- SimplePatternValidator - отклоняет слишком простые пароли (все шаги одинаковой длины);

- ValidationChain - обёртка, которая собирает обработчиков в цепочку.

Цепочка выстраивается в порядке:

Длина → Повторы → Диапазон → Простота

Каждый обработчик возвращает ValidationResult:

- если проверка не пройдена - возвращает ValidationResult(False, message) и прерывает цепочку;

- если проверка пройдена - возвращает None и передаёт запрос следующему обработчику.

Если все обработчики вернули None, последовательность считается корректной.

Таким образом, клиентский код (AppWindow) вызывает только validation_chain.validate(sequence) и не знает, сколько обработчиков в цепочке и в каком порядке они расположены.

2. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter.

Основное окно содержит сетку из девяти скруглённых ячеек, пронумерованных от 1 до 9. Пользователь формирует пароль путём последовательного выбора ячеек.

К паролю предъявляются требования, реализованные в цепочке:

- длина от 4 до 9 ячеек;

- отсутствие повторов;

- все ячейки в диапазоне 1–9;

- пароль не должен быть слишком простым (одинаковый шаг между соседними ячейками).

Приложение содержит два режима: «Создание пароля» и «Вход».

Дополнительно реализованы:

- панель шаблонов («Крест», «Квадрат», «Зигзаг»);

- кнопка «Проверить цепочкой» - запускает валидацию через цепочку обработчиков и показывает результат;

- отображение состава цепочки над кнопками («Цепочка: Проверка длины → Проверка повторов → Проверка диапазона → Проверка простоты»);

- кнопка «Очистить»;

- кнопка «Сбросить сохранённый пароль».

Валидация выполняется только через цепочку, в том числе при сохранении пароля и при входе.

3. Диаграмма классов

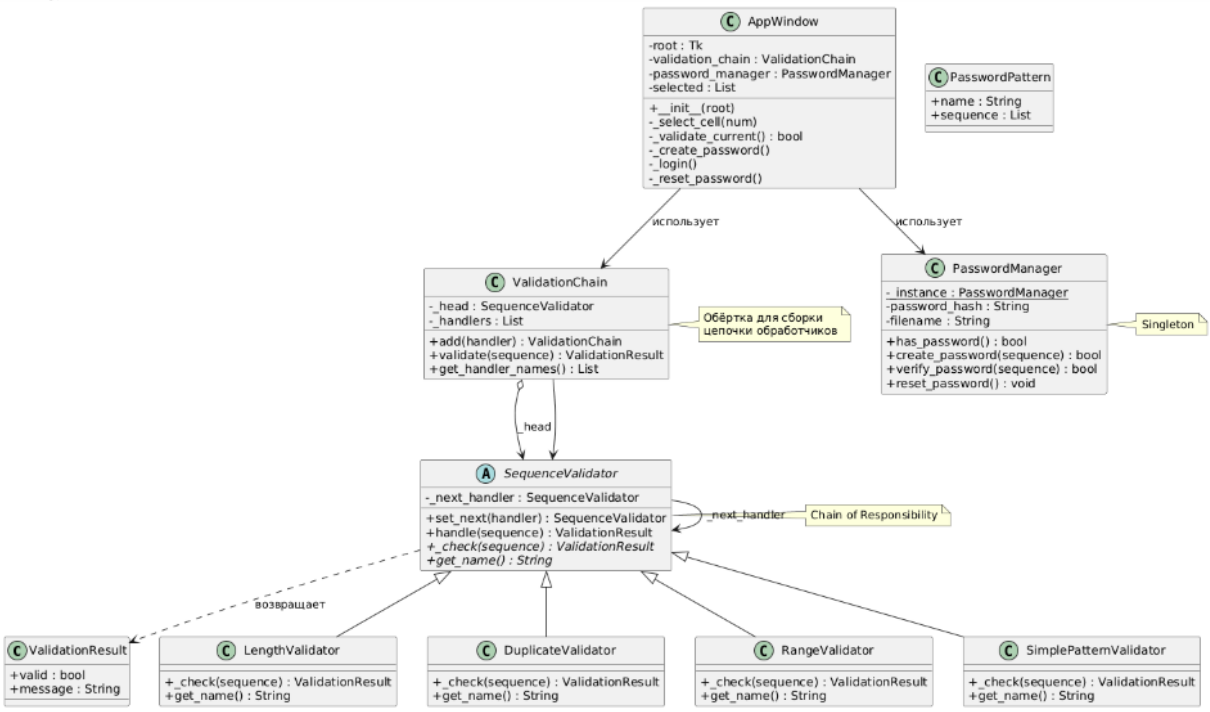


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

Диаграмма содержит:

- SequenceValidator и его наследников (LengthValidator, DuplicateValidator, RangeValidator, SimplePatternValidator) - паттерн «Цепочка обязанностей»;

- ValidationChain - обёртка для сборки цепочки;

- ValidationResult - результат проверки;

- AppWindow, PasswordManager, PasswordPattern - основные классы приложения.

4. Реализация паттерна «Цепочка обязанностей»

Абстрактный обработчик:

```
class SequenceValidator(ABC):
    def __init__(self):
        self._next_handler = None

    def set_next(self, handler):
        self._next_handler = handler
        return handler

    def handle(self, sequence):
        result = self._check(sequence)
        if result is not None:
            return result
        if self._next_handler is not None:
            return self._next_handler.handle(sequence)
        return ValidationResult(True, "")

    @abstractmethod
    def _check(self, sequence):
        pass
```

```
    @abstractmethod
    def get_name(self):
        pass
```

Конкретные обработчики:

```
class LengthValidator(SequenceValidator):
    def get_name(self):
        return "Проверка длины"

    def _check(self, sequence):
        if not isinstance(sequence, list):
            return ValidationResult(False, "Последовательность должна быть списком.")
        if len(sequence) < 4:
            return ValidationResult(False, "Слишком короткая: минимум 4 ячейки.")
        if len(sequence) > 9:
            return ValidationResult(False, "Слишком длинная: максимум 9 ячеек.")
        return None


class DuplicateValidator(SequenceValidator):
    def get_name(self):
        return "Проверка повторов"

    def _check(self, sequence):
        if len(set(sequence)) != len(sequence):
            return ValidationResult(False, "Есть повторяющиеся ячейки.")
        return None
```

```
class RangeValidator(SequenceValidator):
    def get_name(self):
        return "Проверка диапазона"

    def _check(self, sequence):
        for x in sequence:
            if not isinstance(x, int) or x < 1 or x > 9:
                return ValidationResult(False, f"Ячейка {x} вне диапазона 1–
9.")
        return None


class SimplePatternValidator(SequenceValidator):
    def get_name(self):
        return "Проверка простоты"

    def _check(self, sequence):
        if len(sequence) >= 3:
            diffs = [sequence[i+1] - sequence[i] for i in range(len(sequence)-1)]
            if len(set(diffs)) == 1:
                return ValidationResult(
                    False,
                    "Слишком простой пароль: все шаги одинаковые."
                )
        return None
```

Сборка цепочки:

```
class ValidationChain:
    def __init__(self):
        self._head = None
```

```
        self._handlers = []

    def add(self, handler):
        self._handlers.append(handler)
        if self._head is None:
            self._head = handler
        else:
            last = self._handlers[-2]
            last.set_next(handler)
        return self

    def validate(self, sequence):
        if self._head is None:
            return ValidationResult(True, "")
        return self._head.handle(sequence)
```

Сборка цепочки в AppWindow:

```
self.validation_chain = ValidationChain()
self.validation_chain.add(LengthValidator())
self.validation_chain.add(DuplicateValidator())
self.validation_chain.add(RangeValidator())
self.validation_chain.add(SimplePatternValidator())
```

Использование в UI:

```
def _validate_current(self):
    result = self.validation_chain.validate(self.selected)
    if result.valid:
        messagebox.showinfo("Цепочка обязанностей",
"Последовательность корректна.")
    else:
        messagebox.showerror("Цепочка обязанностей", result.message)
```

При сохранении и при входе валидация также выполняется через цепочку - клиент не знает, сколько обработчиков и какие правила они проверяют.

5. Результаты работы программы

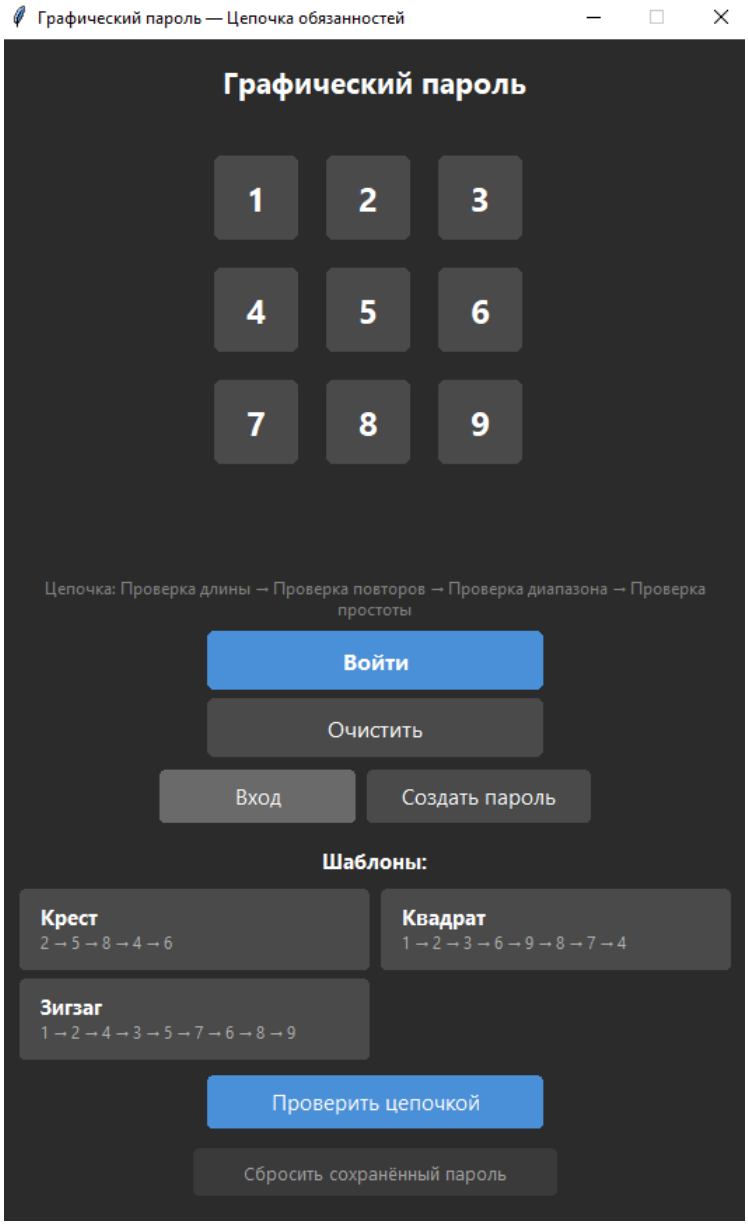


Рисунок 2 - Главное окно приложения

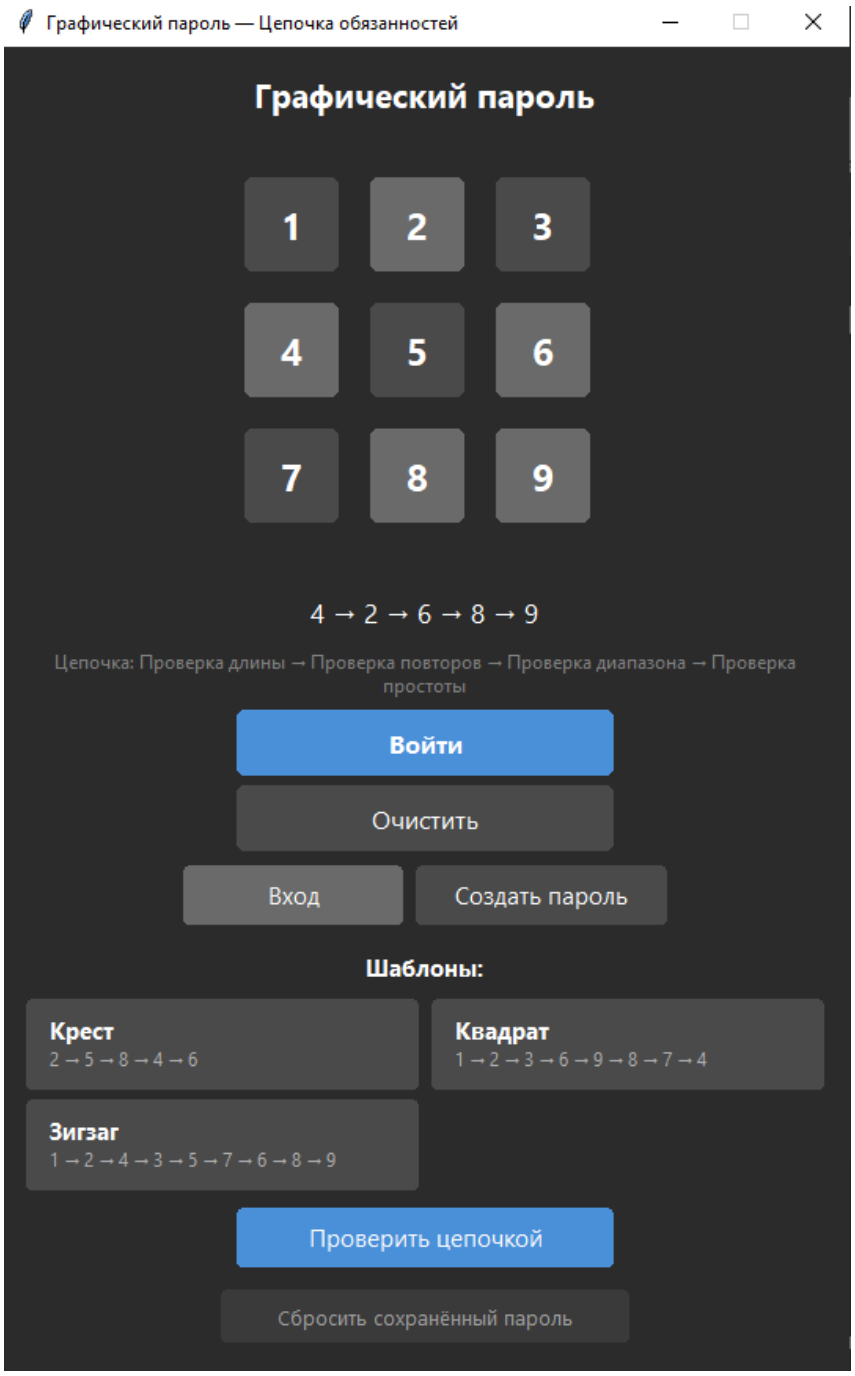


Рисунок 3 - Формирование последовательности

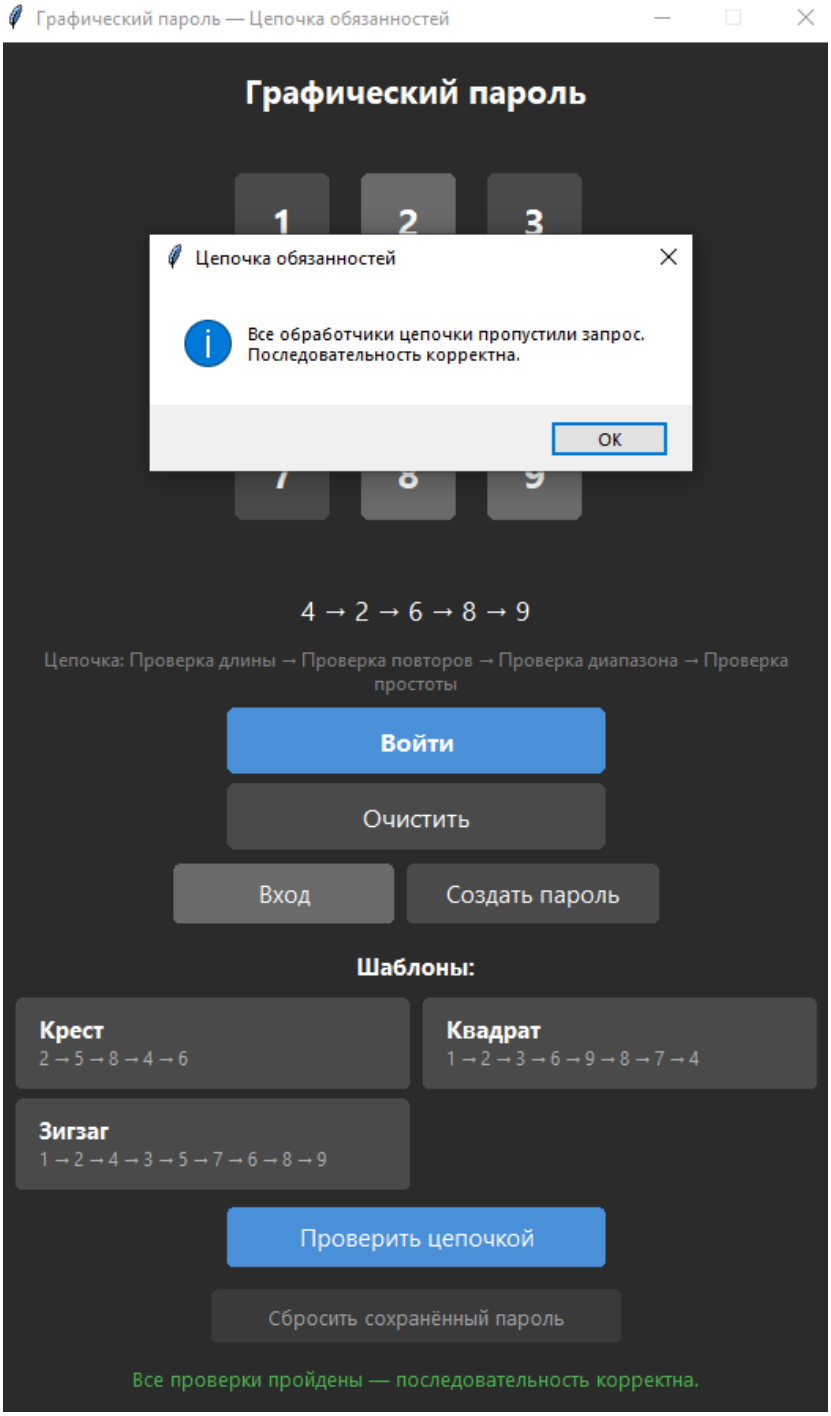


Рисунок 4 - Успешная проверка цепочкой (все обработчики пропустили запрос)

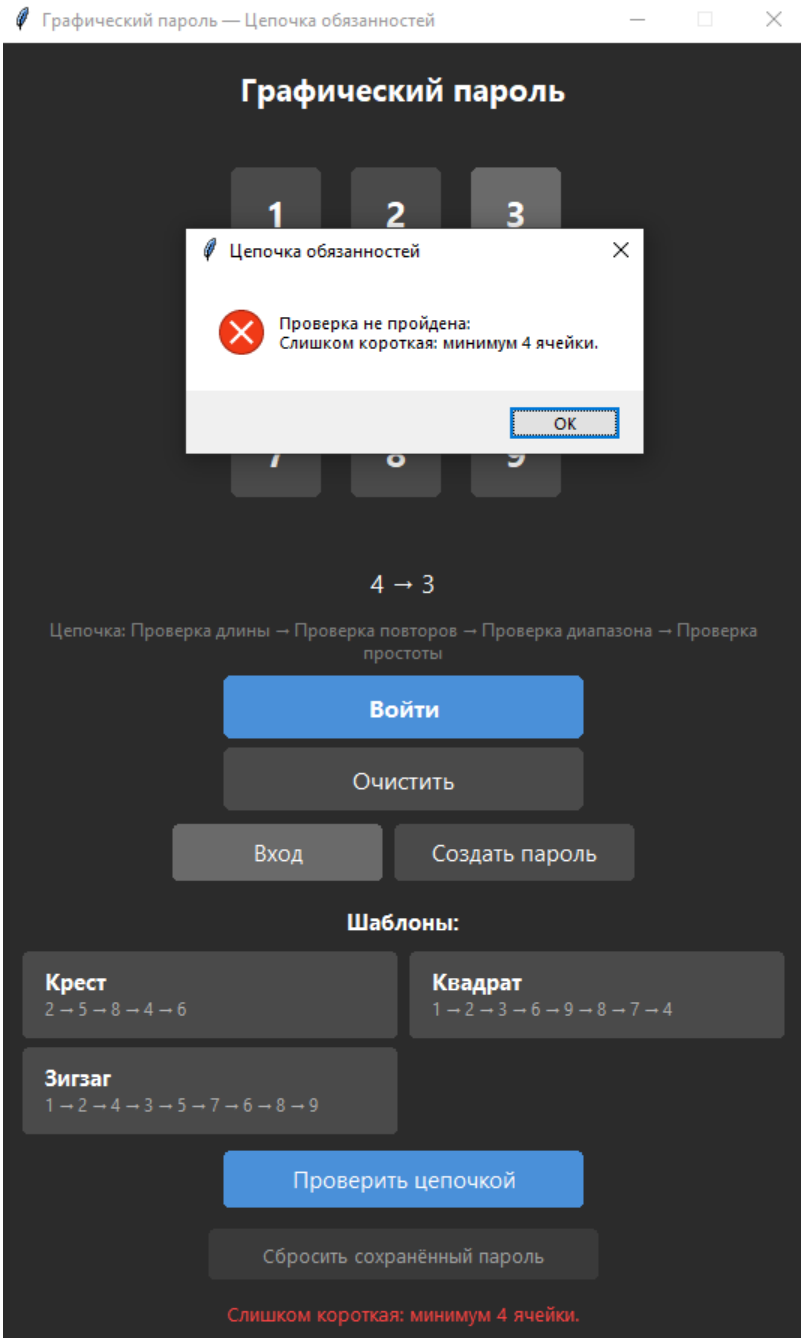


Рисунок 5 - Ошибка проверки длины

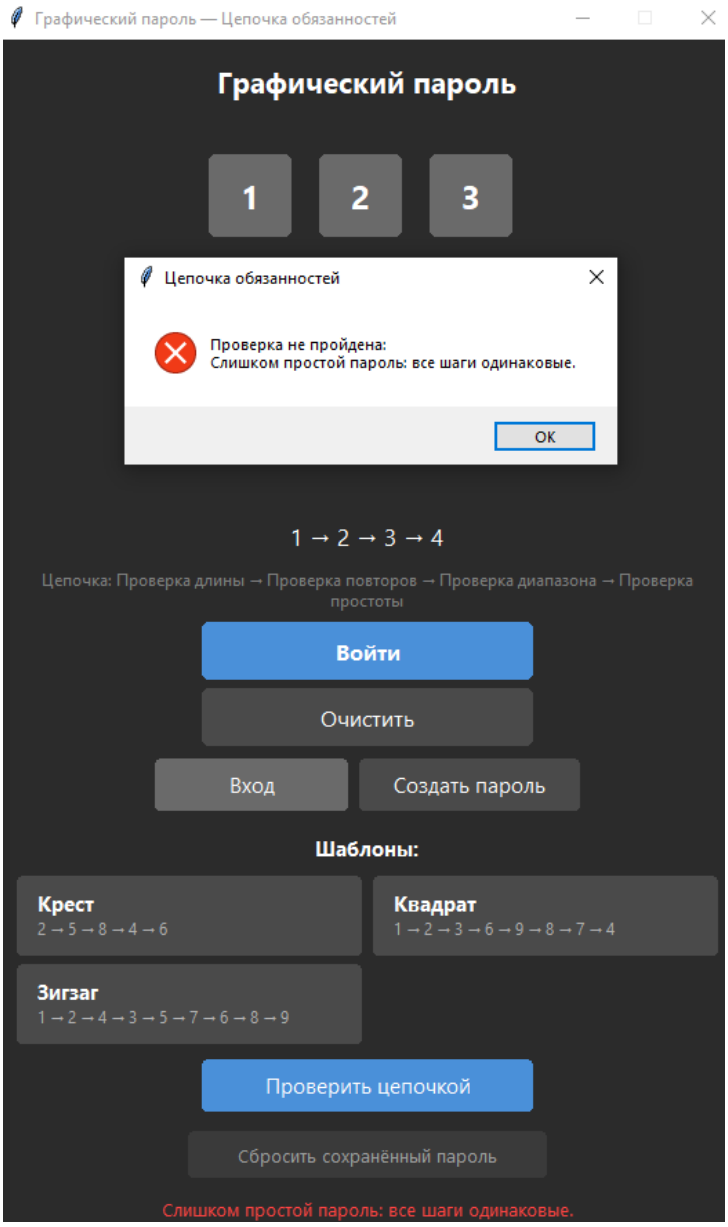


Рисунок 6 - Ошибка проверки простоты пароля

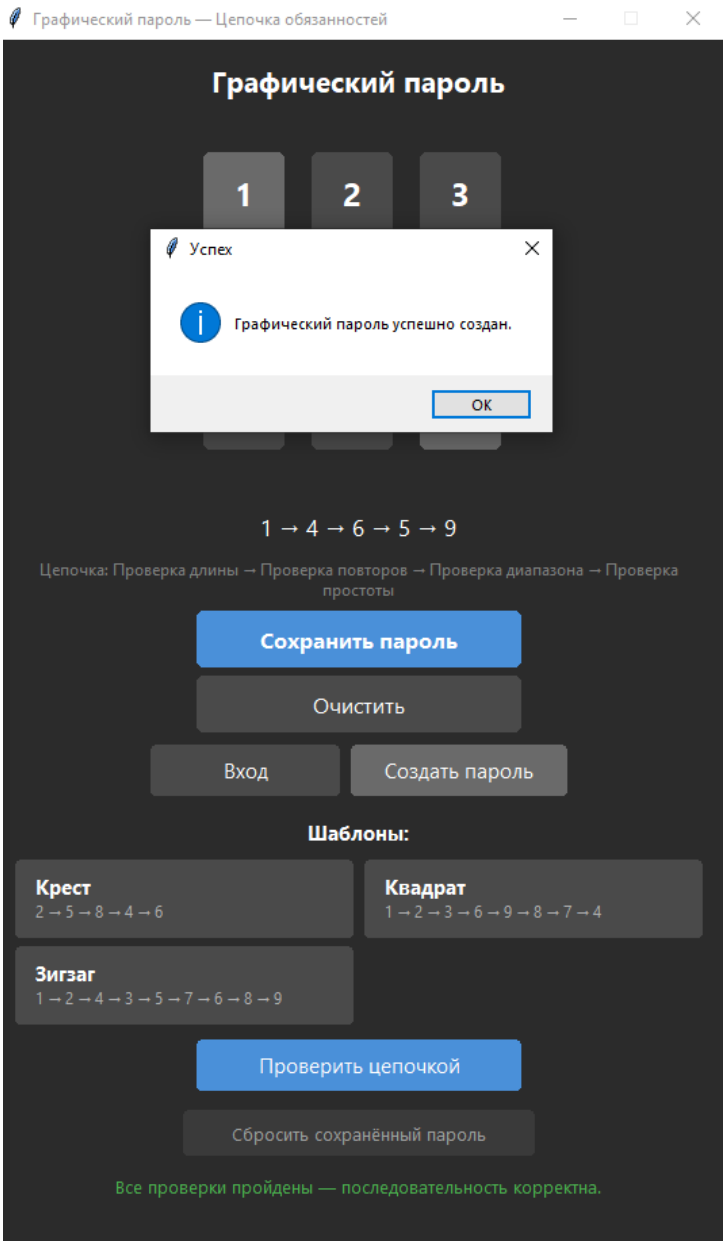


Рисунок 7 - Сохранение пароля (с предварительной валидацией цепочкой)

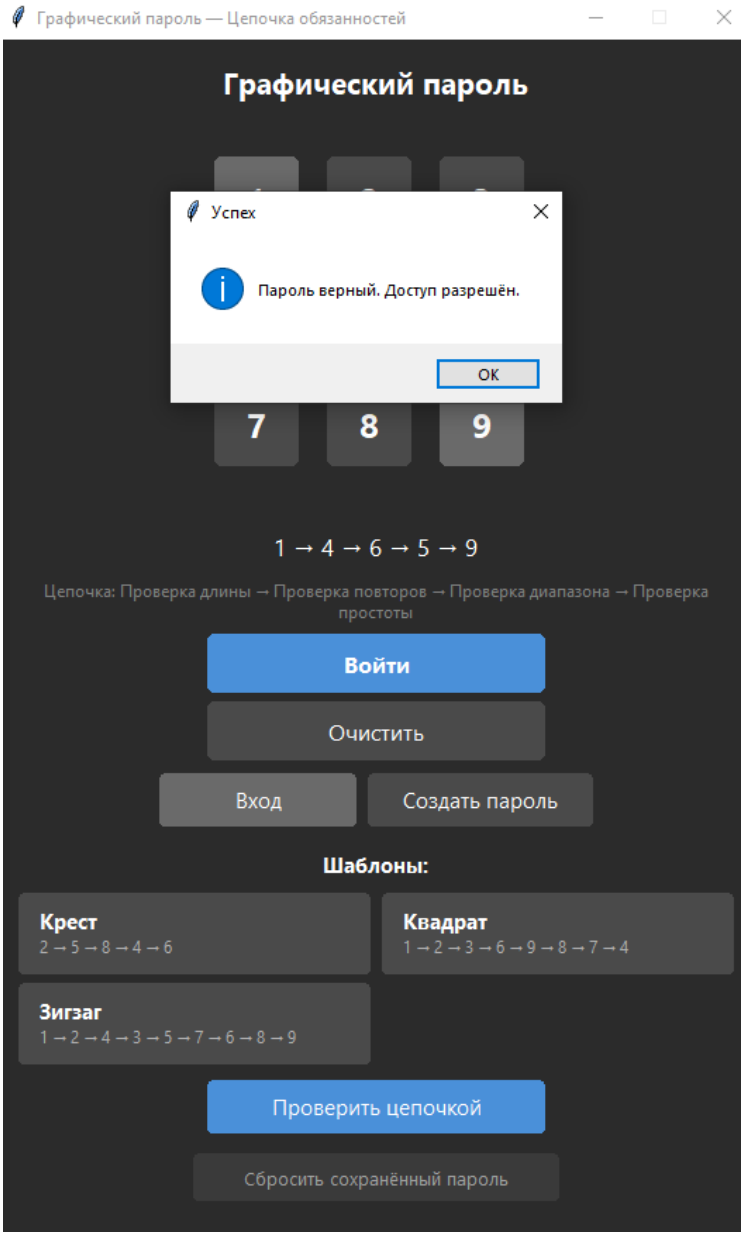


Рисунок 8 - Проверка правильного пароля

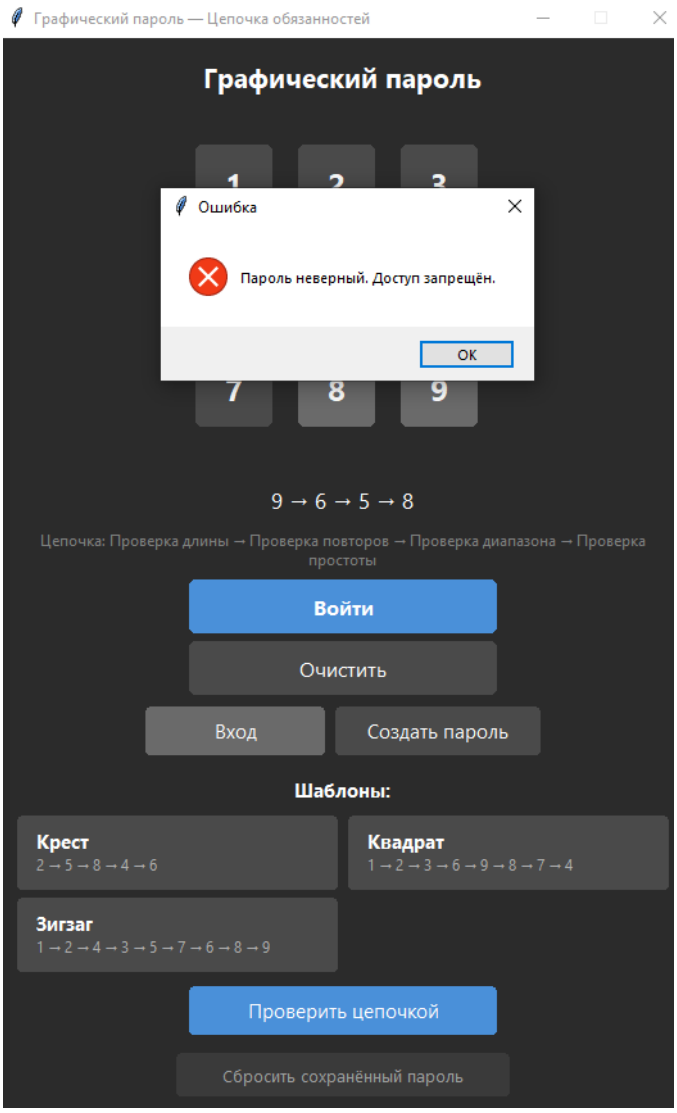


Рисунок 9 - Проверка неправильного пароля

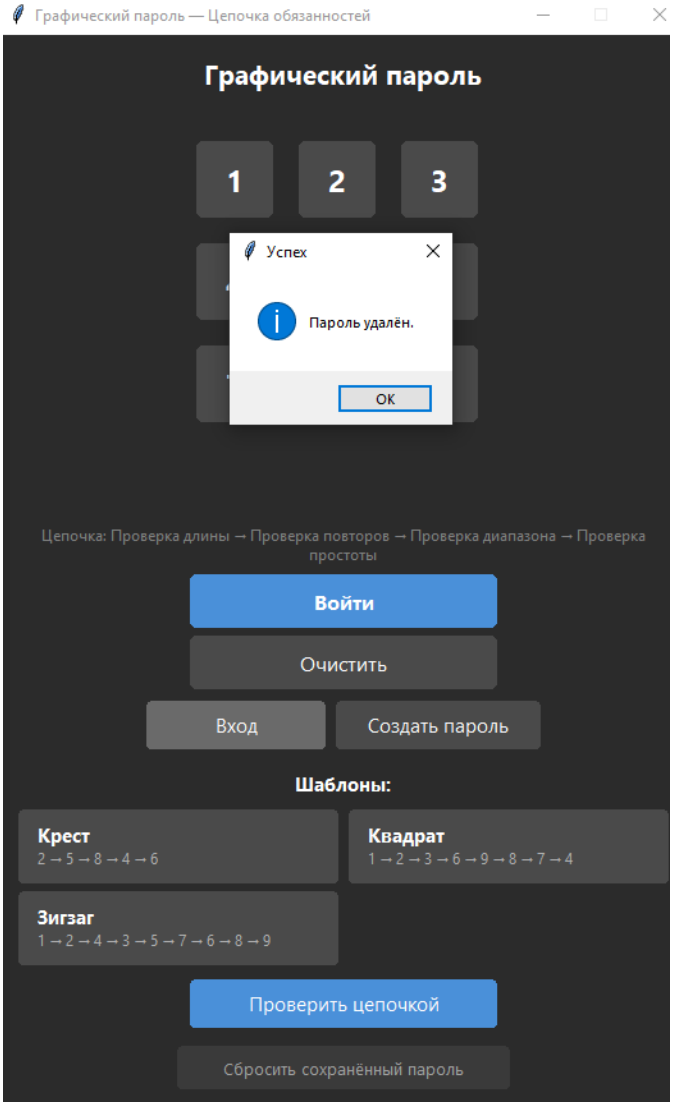


Рисунок 10 - Сброс сохранённого пароля

6. Исходный код

Основные элементы программы:

- SequenceValidator - абстрактный обработчик цепочки;LengthValidator, DuplicateValidator, RangeValidator, SimplePatternValidator - конкретные обработчики;
- ValidationChain - обёртка для сборки цепочки;
- ValidationResult - результат проверки;
- PasswordPattern - шаблон графического пароля;
- PasswordManager - Singleton с хэшированием;
- AppWindow - основной класс интерфейса;
- RoundedButton, rounded_rect() - скруглённые кнопки и ячейки;

– hash_sequence() - вспомогательная функция получения SHA-256 хэша.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Был изучен и реализован поведенческий паттерн проектирования «Цепочка обязанностей» (Chain of Responsibility). Паттерн применён для валидации последовательности графического пароля: каждое правило вынесено в отдельный обработчик (LengthValidator, DuplicateValidator, RangeValidator, SimplePatternValidator), а цепочка собирается в ValidationChain. Обработчики связаны в цепочку и передают запрос последовательно, прерывая его при первой ошибке. Клиентский код (AppWindow) обращается только к validation_chain.validate(sequence) и не знает о внутреннем устройстве цепочки. В результате получена работоспособная программа, демонстрирующая практическое применение поведенческого паттерна «Цепочка обязанностей» при разработке прикладного программного обеспечения.